

Part 1 — Passive HTTP Request Fingerprinting

Initial Passive Header Inspection

Extracting basic baseline HTTP request headers received passively by the Flask server without client-side script execution.

```
=====New visit=====
IP address   : 127.0.0.1
HTTP method  : GET
User-Agent   : GET
User-Agent    : Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36
Accept-Languages : en-US,en;q=0.9
=====
```

Figure 1: Server console logging initial passive request attributes (IP address, HTTP method, User-Agent, and Accept-Languages).

Expanded Passive Collector (Brave Browser)

Expanding header introspection to extract deeper environment signals, including MIME types, Client Hints, and navigation metadata.

```
=====New visit=====
IP address   : 127.0.0.1
HTTP method  : GET
User-Agent   : GET
User-Agent    : Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36
Accept-Language : en-US,en;q=0.9
Accept       : text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8
Sec-CH-UA    : "Chromium";v="154", "Brave";v="154", "Not A(Brand";v="99"
Sec-CH-UA-Platform "Windows"
Sec-Fetch-Site : none
=====
```

Figure 2: Server console capturing additional passive signals (Accept, Sec-CH-UA, Sec-CH-UA-Platform, and Sec-Fetch-Site) on Brave.

Comparative Analysis: Microsoft Edge

Testing the expanded passive collector across a different browser engine family to observe header variance.

```
=====New visit=====
IP address   : 127.0.0.1
HTTP method  : GET
User-Agent   : GET
User-Agent    : Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36 Edge/154.0.0.0
Accept-Language : fr,fr-FR;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
Accept       : text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-CH-UA    : "Chromium";v="154", "Microsoft Edge";v="154", "Not A (Brand";v="99"
Sec-CH-UA-Platform "Windows"
Sec-Fetch-Site : none
=====
```

Figure 3: Server console logging request headers from Microsoft Edge, exposing engine-specific differences in Sec-CH-UA ("Microsoft Edge"), localized Accept-Language weights, and extended Accept MIME types.

Part 2 — Active Feature Collection (JavaScript)

Client-Side Environment Introspection

Querying browser and hardware APIs via client-side JavaScript (`navigator`, `screen`, `Intl`) and rendering the attributes directly in the DOM.



Figure 4: Client webpage displaying actively extracted attributes (Language, TimeZone, Screen Resolution, Device Pixel Ratio, and CPU Cores).

Server-Side Data Ingestion via POST /collect

Transmitting active client-side properties to the Flask backend asynchronously using the Fetch API.

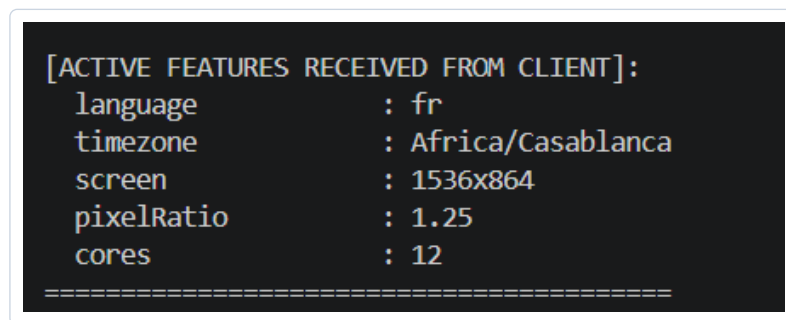


Figure 5: Flask server console displaying structured active environment features received via JSON POST request.

Defense Evaluation: Private / Incognito Mode

Evaluating feature persistence and stability when accessing the application through a Private/Incognito browsing session.

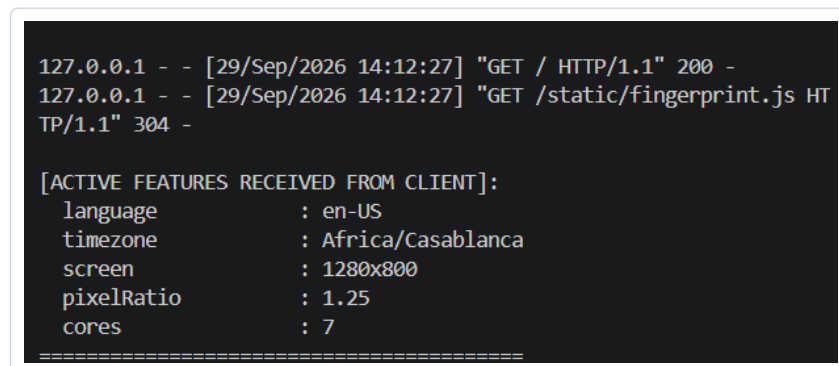


Figure 6: Flask server console confirming active environment features remain fully accessible and identical in private browsing mode.

Analysis & Effectiveness of Protections

Testing across private-browsing mode and cookie blocking reveals that all collected active features (**language**, **screenResolution**, **cores**, **timezone**, **pixelRatio**) remain completely unchanged. This demonstrates that traditional stateful privacy mechanisms only isolate storage and have zero impact on stateless hardware and environment introspection APIs.

In contrast, advanced anti-fingerprinting solutions (such as Tor Browser, Brave's Farbling, or Firefox's **resistFingerprinting**) prove most effective because they enforce attribute uniformity (e.g., standardizing screen dimensions, locking timezones to UTC, and spoofing core counts) across the user base rather than merely blocking cookies. However, this approach faces key limitations: spoofing or restricting core APIs often causes severe website breakage (such as distorted UI layouts, incorrect localized scheduling, and broken graphics rendering), while relying on niche spoofing extensions paradoxically creates a more unique and recognizable fingerprint profile.

Part 3 — Behavioral Fingerprinting (Typing Biometrics)

Client-Side Keystroke & Rhythm Measurement

Capturing dynamic behavioral keystroke dynamics (typing speed, completion duration, and error correction count) during sentence reproduction.

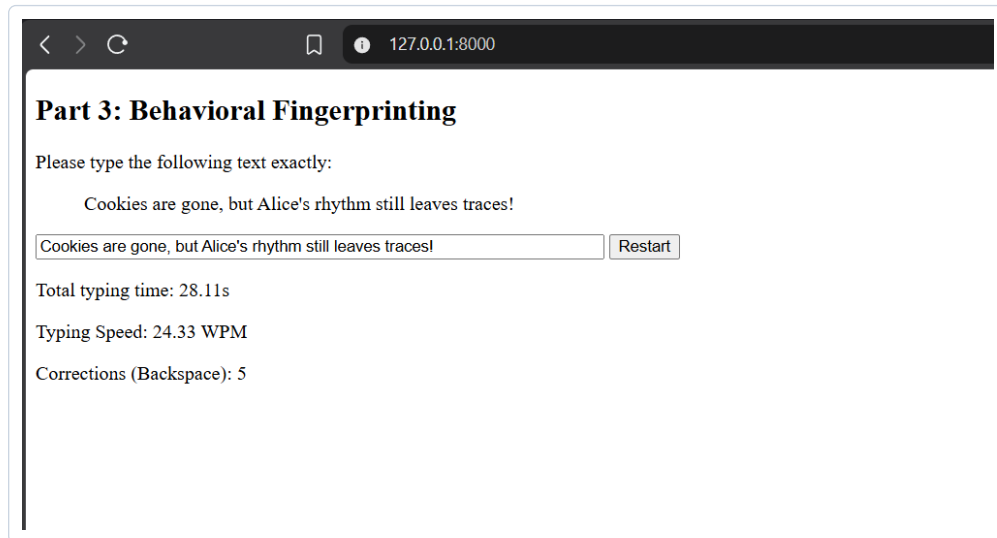


Figure 7: Behavioral UI displaying real-time calculated typing metrics (Total typing time, Typing Speed in WPM, and Backspace corrections).

Server-Side Combined Fingerprint Ingestion

Consolidating static active environment attributes with dynamic behavioral typing metrics into a single unified JSON payload.

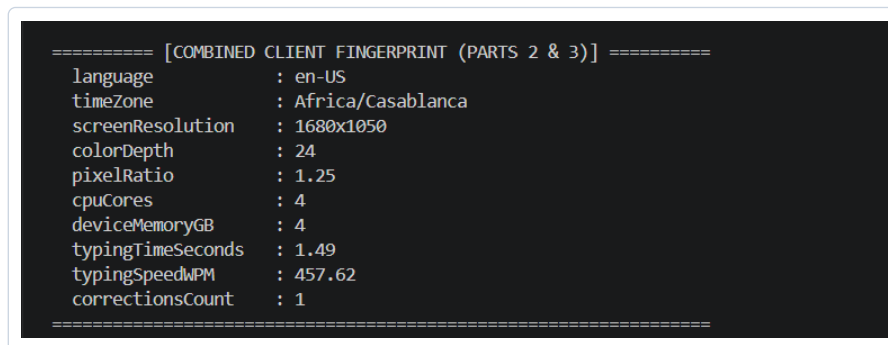


Figure 8: Flask server terminal logging the comprehensive fingerprint combining active device properties and user typing biometrics.

Part 4 — Identifier Generation (SHA-256 Hashing)

Cryptographic Fingerprint Generation (crypto.subtle)

Hashing combined fingerprint attributes on the client side using SHA-256 and transmitting only the unique 64-character hash identifier to the backend.

```

usser/Desktop/s3/4-infoSec/TP-4/fingerprintinglab/lab4infoSecEnv/scripts/python.exe C:/ossec
ussef/Desktop/s3/4-infoSec/TP-4/fingerprintinglab/app.py
fingerprint      : 3d21e1e4d912431afe550c04652d62a7213845508bd89499bd0a9a1675226849
=====

127.0.0.1 - - [02/Oct/2026 22:54:40] "POST /collect HTTP/1.1" 200 -

===== [COMBINED CLIENT FINGERPRINT (PARTS 2 & 3)] =====
fingerprint      : bf177ba886a2c96c5bcc150b448533b1d1d57816ea5c48fb8c3d705e8f376bb6
=====

127.0.0.1 - - [02/Oct/2026 22:55:05] "POST /collect HTTP/1.1" 200 -

===== [COMBINED CLIENT FINGERPRINT (PARTS 2 & 3)] =====
fingerprint      : 7e9dacc931d3b0b4df2aade1ab7e7eb3a0c1d4aa9f166f6c3c2306da64a5965
=====

```

*Figure 9: Server console logging incoming SHA-256 fingerprint identifiers (*fingerprint*), illustrating hash variations caused by dynamic behavioral biometric inputs across consecutive typing sessions.*